

---

**NAME:** S.DHARSHINI

**REG NO:** 192424258

**SUB CODE:** DSA0502

**SUB NAME:** QUERY PROCESSING FOR DATA SCIENCE

**CO2 ASSESSMENT TOOL 1**

---

### **Scenario 5: Hotel Reservation and Booking System**

A hotel chain operates multiple branches across different cities. Customers can book rooms online or through customer service representatives. The hotel management wants a centralized system to manage customer information, room availability, reservations, payments, and employee details. Different room categories such as standard, deluxe, and suite must be maintained. The system should track check-in and check-out details, generate invoices, and maintain customer booking history. Analyze the requirements and design a database structure that minimizes redundancy and supports efficient querying. Use Python to connect to the database and demonstrate room booking, reservation updates, and customer management functionalities.

**AIM:** To design and implement a Hotel Reservation and Booking System using MySQL and Python to manage hotel branches, customers, room reservations, payments, employees, and booking history efficiently.

**PROBLEM STATEMENT:** Develop a centralized hotel reservation system that stores customer details, room availability, reservations, payments, employee information, and booking history while minimizing redundancy through database normalization.

## PROCEDURE:

- Connect Python to the MySQL database.
- Create the required database and tables.
- Insert sample records into each table.
- Perform customer registration and room booking.
- Update or cancel reservations as required.
- Display booking details, payment information, and customer booking hist

## OUTPUT:

The output shows the MySQL Command Line Client interface with the following commands and results:

```
mysql> CREATE DATABASE hoteldb;
Query OK, 1 row affected (0.134 sec)

mysql> USE hoteldb;
Database changed

mysql> CREATE TABLE Branch
  -> BranchID INT PRIMARY KEY AUTO_INCREMENT,
  -> BranchName VARCHAR(100),
  -> City VARCHAR(100),
  -> Address VARCHAR(200),
  -> Phone VARCHAR(15)
  -> ;
Query OK, 0 rows affected (1.287 sec)

mysql> desc Branch;
+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| BranchID | int | NO | PRI | NULL | auto_increment |
| BranchName | varchar(100) | YES | | NULL | |
| City | varchar(100) | YES | | NULL | |
| Address | varchar(200) | YES | | NULL | |
| Phone | varchar(15) | YES | | NULL | |
+-----+-----+-----+-----+-----+
5 rows in set (0.421 sec)

mysql> CREATE TABLE Customer
  -> CustomerID INT PRIMARY KEY AUTO_INCREMENT,
  -> CustomerName VARCHAR(100),
  -> Phone VARCHAR(15),
  -> Email VARCHAR(100),
  -> Address VARCHAR(200)
  -> ;
Query OK, 0 rows affected (0.487 sec)

mysql> desc Customer;
+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| CustomerID | int | NO | PRI | NULL | auto_increment |
| CustomerName | varchar(100) | YES | | NULL | |
| Phone | varchar(15) | YES | | NULL | |
+-----+-----+-----+-----+-----+
3 rows in set (0.471 sec)

mysql> CREATE TABLE Reservation
  -> ReservationID INT PRIMARY KEY AUTO_INCREMENT,
  -> CustomerID INT,
  -> RoomID INT,
  -> CheckIn DATE,
  -> CheckOut DATE,
  -> BookingDate DATE,
  -> Status VARCHAR(10),
  -> FOREIGN KEY(CustomerID) REFERENCES Customer(CustomerID),
  -> FOREIGN KEY(RoomID) REFERENCES Room(RoomID)
  -> ;
Query OK, 0 rows affected (0.406 sec)

mysql> desc Reservation;
+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| ReservationID | int | NO | PRI | NULL | auto_increment |
| CustomerID | int | YES | | NULL | |
| RoomID | int | YES | | NULL | |
| CheckIn | date | YES | | NULL | |
| CheckOut | date | YES | | NULL | |
| BookingDate | date | YES | | NULL | |
| Status | varchar(10) | YES | | NULL | |
+-----+-----+-----+-----+-----+
7 rows in set (0.471 sec)

mysql> CREATE TABLE Payment
  -> PaymentID INT PRIMARY KEY AUTO_INCREMENT,
  -> ReservationID INT,
  -> Amount DECIMAL(10,2),
  -> PaymentMethod VARCHAR(50),
  -> PaymentDate DATE,
  -> FOREIGN KEY(ReservationID) REFERENCES Reservation(ReservationID)
  -> ;
Query OK, 0 rows affected (0.406 sec)

mysql> desc Payment;
+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| PaymentID | int | NO | PRI | NULL | auto_increment |
| ReservationID | int | YES | | NULL | |
| Amount | decimal(10,2) | YES | | NULL | |
| PaymentMethod | varchar(50) | YES | | NULL | |
| PaymentDate | date | YES | | NULL | |
+-----+-----+-----+-----+-----+
5 rows in set (0.457 sec)

mysql> CREATE TABLE Employee
  -> EmployeeID INT PRIMARY KEY AUTO_INCREMENT,
  -> EmployeeName VARCHAR(100),
  -> Designation VARCHAR(100),
  -> Salary DECIMAL(10,2),
  -> BranchID INT,
  -> FOREIGN KEY(BranchID) REFERENCES Branch(BranchID)
  -> ;
Query OK, 0 rows affected (0.411 sec)

mysql> desc Employee;
+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| EmployeeID | int | NO | PRI | NULL | auto_increment |
| EmployeeName | varchar(100) | YES | | NULL | |
| Designation | varchar(100) | YES | | NULL | |
| Salary | decimal(10,2) | YES | | NULL | |
| BranchID | int | YES | | NULL | |
+-----+-----+-----+-----+-----+
5 rows in set (0.418 sec)

mysql> SHOW TABLES;
+-----+
| Tables_in_hoteldb |
+-----+
| branch |
| customer |
| employee |
| payment |
| reservation |
| room |
+-----+
6 rows in set (0.155 sec)

mysql> INSERT INTO Branch(BranchName, City, Address, Phone)
  -> VALUES
  -> ('Channel Grand', 'Channel', 'Anna Nagar', '9876543210'),
  -> ('Celebration Inn', 'Chennai', '98 Puram', '9876543210');
Query OK, 2 rows affected (0.192 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Customer(CustomerName, Phone, Email, Address)
  -> VALUES
  -> ('Rishi', '9876543211', 'rishi@gmail.com', 'Chennai'),
  -> ('Priya', '9876543212', 'priya@gmail.com', 'Madurai');
Query OK, 2 rows affected (0.111 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Room(BranchID, RoomType, Price, Availability)
  -> VALUES
  -> (1, 'Standard', '900', 'Available'),
  -> (1, 'Deluxe', '1200', 'Available'),
  -> (2, 'Suite', '2000', 'Available');
```

The image shows two side-by-side screenshots of a MySQL 5.6 Command Line Client window. The left screenshot displays several SQL queries and their results:

- `mysql> SELECT * FROM Branch;` Results in a table with 3 rows: Chetana Grand, Chetana Grand, and Chetana Grand.
- `mysql> SELECT * FROM Customer;` Results in a table with 5 rows: Ravi, Priya, Ravi, Priya, and Ravi.
- `mysql> SELECT * FROM Room;` Results in a table with 4 rows: Standard, Deluxe, Suite, and Suite.
- `mysql> USE HotelDB;`
- `mysql> SELECT * FROM Reservation;` Results in a table with 1 row: 1, 1, 2020-08-05, 2020-08-08, 2020-07-10, Checked in.

The right screenshot shows the same client with the following queries and results:

- `mysql> SELECT * FROM Customer;` Results in a table with 5 rows: Ravi, Priya, Ravi, Priya, and Ravi.
- `mysql> SELECT * FROM Reservation;` Results in a table with 1 row: 1, 1, 2020-08-05, 2020-08-08, 2020-07-10, Checked in.
- `mysql> SELECT * FROM Room;` Results in a table with 4 rows: Standard, Deluxe, Suite, and Suite.
- `mysql> SELECT * FROM Reservation;` Results in a table with 1 row: 1, 1, 2020-08-05, 2020-08-08, 2020-07-10, Checked in.

**CONCLUSION:** The system provides a centralized solution for hotel management by reducing data redundancy, improving reservation handling, maintaining room availability, and enabling efficient querying through a well-designed relational database.

**Scenario 10: Smart Transportation and Ticket Booking System**

A transportation company operates buses across multiple cities. Passengers can search routes, book tickets, cancel reservations, and make online payments. The company must maintain information about buses, routes, schedules, drivers, passengers, and ticket bookings. Each bus operates on multiple routes, and passengers may make several bookings over time. Management wants to analyze passenger demand, route utilization, and revenue generation. Design a database system that efficiently stores transportation data and supports ticket booking operations. Use Python SQL libraries to connect to the database and perform CRUD operations for route management, passenger registration, and ticket reservations.

**AIM:** To design and implement a Smart Transportation and Ticket Booking System using MySQL and Python for managing buses, routes, drivers, passengers, ticket bookings, schedules, and payments while supporting efficient CRUD operations.

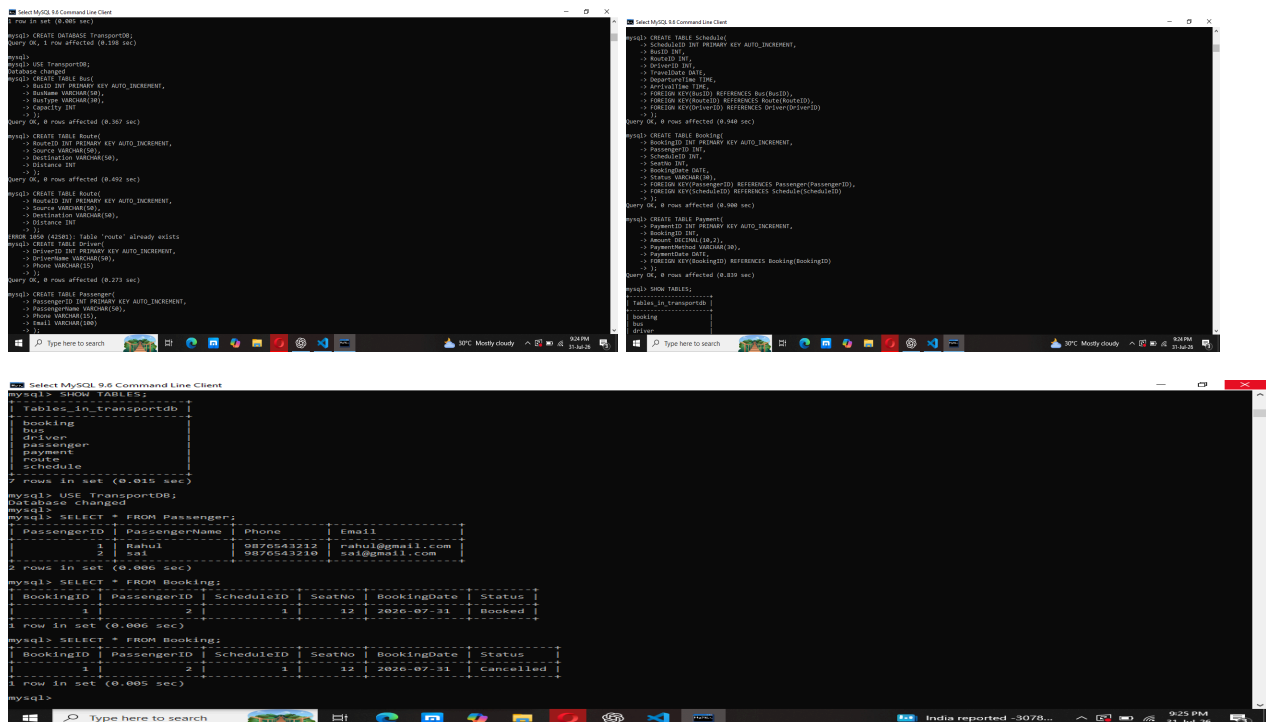
**PROBLEM STATEMENT:** A transportation company operates buses across multiple cities. Passengers need to search routes, book tickets, cancel reservations, and make online payments.

The company also needs to maintain details of buses, routes, schedules, drivers, passengers, and bookings. The system should support efficient storage of transportation data, reduce redundancy, and provide quick access to booking and passenger information through a centralized database.

## PROCEDURE:

- Open MySQL and create the TransportDB database.
- Create all required tables with primary and foreign keys.
- Insert sample records into the Bus, Route, Driver, Passenger, and Schedule tables.
- Open the project folder in VS Code.
- Connect Python to the MySQL database using **mysql.connector**.
- Execute the Passenger Registration program.
- Execute the Ticket Booking program.
- Update booking details using the Update Booking module.
- Display booking details using SQL JOIN queries.
- Verify the output and database records.

## OUTPUT:



```
mysql> CREATE DATABASE TransportDB;
Query OK, 1 row affected (0.108 sec)

mysql> USE TransportDB;
Database changed

mysql> CREATE TABLE Bus
-> CREATE TABLE `Bus` (
  `BusID` INT(11) UNSIGNED NOT NULL AUTO_INCREMENT,
  `BusName` VARCHAR(50) NOT NULL,
  `Capacity` INT(11) UNSIGNED NOT NULL,
  PRIMARY KEY (`BusID`),
  INDEX (`BusName`)) ENGINE=InnoDB;
Query OK, 0 rows affected (0.367 sec)

mysql> CREATE TABLE Route
-> CREATE TABLE `Route` (
  `RouteID` INT(11) UNSIGNED NOT NULL AUTO_INCREMENT,
  `Source` VARCHAR(50) NOT NULL,
  `Destination` VARCHAR(50) NOT NULL,
  `Distance` INT(11) UNSIGNED NOT NULL,
  PRIMARY KEY (`RouteID`),
  INDEX (`Source`),
  INDEX (`Destination`)) ENGINE=InnoDB;
Query OK, 0 rows affected (0.492 sec)

mysql> CREATE TABLE Driver
-> CREATE TABLE `Driver` (
  `DriverID` INT(11) UNSIGNED NOT NULL AUTO_INCREMENT,
  `DriverName` VARCHAR(50) NOT NULL,
  `Phone` VARCHAR(15) NOT NULL,
  PRIMARY KEY (`DriverID`),
  INDEX (`DriverName`)) ENGINE=InnoDB;
Query OK, 0 rows affected (0.273 sec)

mysql> CREATE TABLE Passenger
-> CREATE TABLE `Passenger` (
  `PassengerID` INT(11) UNSIGNED NOT NULL AUTO_INCREMENT,
  `PassengerName` VARCHAR(50) NOT NULL,
  `Phone` VARCHAR(15) NOT NULL,
  `Email` VARCHAR(50) NOT NULL,
  PRIMARY KEY (`PassengerID`),
  INDEX (`PassengerName`),
  INDEX (`Phone`),
  INDEX (`Email`)) ENGINE=InnoDB;
Query OK, 0 rows affected (0.433 sec)

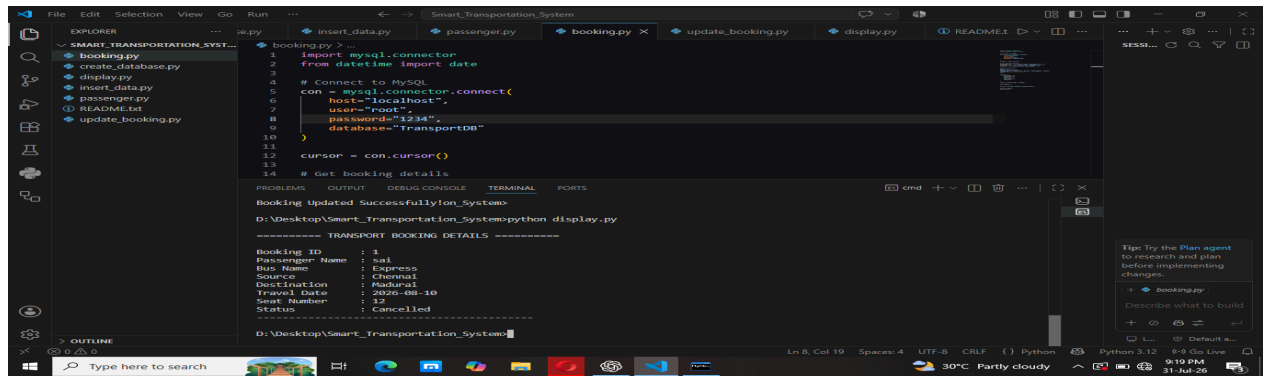
mysql> SHOW TABLES;
+-----+
| Tables_in_TransportDB |
+-----+
| booking                |
| bus                     |
| driver                 |
| passenger              |
| payment                |
| route                  |
| schedule               |
+-----+

mysql> USE TransportDB;
Database changed

mysql> SELECT * FROM Passenger;
+-----+-----+-----+-----+
| PassengerID | PassengerName | Phone          | Email          |
+-----+-----+-----+-----+
| 1           | Rahul        | 9876543212    | rahul@gmail.com |
+-----+-----+-----+-----+
2 rows in set (0.006 sec)

mysql> SELECT * FROM Booking;
+-----+-----+-----+-----+-----+-----+
| BookingID | PassengerID | ScheduleID | SeatNo | BookingDate | Status |
+-----+-----+-----+-----+-----+-----+
| 1         | 1           | 1          | 12     | 2020-07-31 | Booked |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.006 sec)

mysql> SELECT * FROM Booking;
+-----+-----+-----+-----+-----+-----+
| BookingID | PassengerID | ScheduleID | SeatNo | BookingDate | Status |
+-----+-----+-----+-----+-----+-----+
| 1         | 1           | 1          | 12     | 2020-07-31 | Cancelled |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.005 sec)
```



**CONCLUSION:** The Smart Transportation and Ticket Booking System provides a centralized and efficient solution for managing transportation services. It enables easy passenger registration, route management, ticket booking, and booking updates while ensuring data consistency through a relational database. Python and MySQL together provide a simple and effective platform for implementing the system.